

Transaction Isolation Levels

Transaction isolation level-ləri sadə dildə, PostgreSQL davranışı və ECafe rezervasiya flow-u ilə izah edən praktik sənəd.

Əsas fikir

Transaction bir neçə bazadakı əməliyyatı bir paket kimi saxlayır. Isolation level isə eyni anda gələn başqa transaction-ların bir-birini nə qədər görəcəyini və təsir edəcəyini təyin edir.

Bu PDF nəyi cavablandırır?

- Niyə payment instruction endpoint-ində ReadCommitted istifadə edilir?
- Nə zaman Serializable lazımdır və niyə həmişə ən yaxşı seçim deyil?
- Dirty read, non-repeatable read, phantom read və lost update nədir?
- Reservation, masa lock-u, audit outbox və notification transaction daxilində necə birlikdə işləyir?

Praktik kontekst

Bu sənəddə nümunələr ECafe sisteminin iki real flow-u üzərində qurulub: eyni masanın paralel rezervasiyası və menecerin müştəriyə payment instruction göndərməsi.

Hazırlanma tarixi: 18 sentyabr 2026

1. Əvvəlcə transaction nədir?

Transaction - bir biznes əməliyyatına aid bir neçə SQL addımının ya hamısının uğurlu olması, ya da heç birinin qalmamasıdır. Bunun adı atomiklikdir.

Payment instruction flow	Transaction olmasa	Transaction ilə
1. Instruction yaz	Yazılır	Yazılır
2. Müştəri notification-u yarat	Xəta verə bilər	Xəta olsa hamısı geri alınır
3. Audit outbox event yaz	Yazılmaya bilər	Xəta olsa hamısı geri alınır
Nəticə	Yarımçıq biznes vəziyyəti	Tam və ardıcıl vəziyyət

ECafe payment instruction endpoint-ində bu üç addım eyni transaction daxilindədir. Commit yalnız hamısı uğurlu olduqda edilir.

ECafe: eyni transaction daxilində yazma

```
await using var transaction = await _transactionFactory.BeginTransactionAsync(
    IsolationLevel.ReadCommitted, cancellationToken);

await _reservationPaymentInstructionRepository.Add(instruction);
await _notificationService.CreateAsync(notification);
await _auditLogService.RecordRestaurantActionAsync(...);

await transaction.CommitAsync(cancellationToken);
```

Bu kodun isolation level seçimi vacibdir. Transaction var demək, avtomatik olaraq bütün paralel problemlərin həll olunması demək deyil.

Qısa fərq

Transaction: dəyişikliklərin birlikdə commit olunması. Isolation: paralel transaction-ların bir-birinin məlumatını necə görməsi.

2. Paralel işləmədə yaranan əsas problemlər

İki istifadəçi və ya iki worker eyni anda işləyəndə database hansı məlumatı kimə göstərməlidir? Isolation level bunun qaydasını müəyyən edir.

Problem	Sadə izah	ECafe nümunəsi
Dirty read	Commit olunmamış məlumatı oxumaq.	Menecer hələ rollback edəcəyi instruction-u müştəri görür.
Non-repeatable read	Eyni transaction içində eyni sətiri ikinci dəfə oxuyanda fərqli nəticə almaq.	İlk oxunuşda depozit 100 AZN, sonra başqa əməliyyat onu 120 AZN edir.
Phantom read	Eyni filter sorğusunda sonradan yeni sətirlər görünməsi.	Boş masa sayı yoxlanır, paralel rezervasiya yaranır və ikinci yoxlamada nəticə dəyişir.
Lost update	İki əməliyyat bir-birinin update-ni üstələyir.	İki menecer eyni statusu fərqli istiqamətdə dəyişir; son yazan birincinin işini itirir.
Write skew	Fərqli sətirləri oxuyan iki qərar birlikdə biznes qaydasını pozur.	İki sorğu eyni anda masa boşdur deyə düşünür və ikisi də rezervasiya yaradır.

Vacib nüans

ReadCommitted dirty read-in qarşısını alır. Amma eyni biznes resursu üçün iki paralel qərarın yaranmasının qarşısını təkbaşına almır. Masa rezervasiyasında buna görə ayrıca lock istifadə olunur.

3. Dörd standart isolation level

SQL standartında dörd səviyyə var. Səviyyə yüksəldikcə paralel davranış daha sərt olur, amma lock, retry və performans xərci də artır.

Səviyyə	Nəyi qoruyur?	Nə hələ mümkündür?	PostgreSQL qeydi
Read Uncommitted	Praktik olaraq PostgreSQL-də ReadCommitted kimidir.	Non-repeatable və phantom read konseptual olaraq mümkündür.	PostgreSQL dirty read vermir.
Read Committed	Yalnız commit olunmuş data oxunur. Dirty read yoxdur.	Sonrakı statement fərqli committed data görə bilər.	PostgreSQL default-dur.
Repeatable Read	Transaction-un oxu görüntüsü sabit qalır.	Bəzi serializasiya anomaliyaları hələ ola bilər; konflikt transaction abort ola bilər.	Snapshot əsaslı davranış.
Serializable	Nəticə sanki transaction-lar növbə ilə işləyibmiş kimi olmalıdır.	Biznes konfliktində database transaction-u abort edib retry tələb edə bilər.	Ən güclü, ən bahalı seçim.

Bunları 'pisdən yaxşıya' kimi görmək səhvdir. Düzgün seçim odur ki, konkret biznes qaydasını minimal xərcə qorusun.

4. ReadCommitted - ECafe payment instruction üçün niyə doğrudur?

Payment instruction flow-da bir neçə instruction göndərmək biznesə icazəlidir: menecer bir kartdan digər karta keçə, mətnə düzəliş edə və yenisini göndərə bilər. Deməli 'yalnız bir instruction yaransın' qaydası yoxdur.

Yoxlama	ReadCommitted altında nəticə
Commit olunmamış instruction oxunur?	Xeyr. Dirty read yoxdur.
Instruction, notification və audit birlikdə yazılır?	Bəli. Transaction atomikdir.
İki menecer fərqli instruction göndərə bilər?	Bəli, bu biznes baxımından normal tarixçədir.
Masa iki dəfə rezerv edilə bilər?	Bu endpoint-in problemi deyil; rezervasiya flow-u ayrıca lock ilə həll edir.

Düzgün səbəb

ReadCommitted seçimi 'zəif seçim' olduğu üçün deyil. Burada lazımı təminatı verir: yarımçıq data görünmür, üç yazma atomikdir və icazəli paralelliyi lazımsız bloklamır.

Bu flow-da Serializable seçsək, paralel instruction göndərişləri lazımsız serialization failure ala bilər. Sonra client və ya backend retry etməlidir. Halbuki iki instruction-un yaranması bizə problem yaratmır.

5. Masa rezervasiyası başqa tip problemdir

Masa rezervasiyasında qayda fərqlidir: eyni masa, eyni rezervasiya vaxtı üçün bir bloklayıcı rezervasiya ola bilər. Burada paralel iki sorğu eyni anda 'masa boşdur' nəticəsini görərsə, problem yaranır.

Bu qayda isolation level-i hər yerdə Serializable etməklə yox, konkret resursu qoruyan lock ilə həll edilir.

ECafe: masa üçün hədəfli lock

```
await using var transaction = await _transactionFactory.BeginTransactionAsync(
    IsolationLevel.ReadCommitted, cancellationTokens);

await _tableRepository.AcquireReservationLockAsync(
    restaurantId, tableId, cancellationTokens);

await EnsureTableIsAvailableAsync(restaurantId, tableId, reservedAt);
await _reservationRepository.Add(reservation);
await transaction.CommitAsync(cancellationTokens);
```

Yanaşma	Nəticə	Nə vaxt seçilir?
ReadCommitted + konkret table lock	Yalnız eyni masaya aid sorğular növbələnir.	ECafe rezervasiya flow-u üçün ən məqsədli həll.
Serializable	Database daha geniş konflikt izləyir, abort/retry verə bilər.	Qaydanı lock və constraint ilə ifadə etmək çətinləşir.
Heç nə	Race condition ilə iki rezervasiya yarana bilər.	Heç vaxt qəbul edilən seçim deyil.

Yadda saxla

Isolation level bütün sistemi qorumalı bir kilid deyil. Ən yaxşı dizayn: business invariant-i müəyyən et, sonra constraint, lock, unique index, idempotency və transaction vasitələrindən uyğun olanı seç.

6. Serializable nə zaman lazımdır?

Serializable database-ə deyir: nəticə sanki paralel transaction-lar bir-birinin ardınca işləyib. Database bunu təmin edə bilmirsə, onlardan birini serialization failure ilə dayandırır.

Bu error bug deyil. Bu, database-in biznes qaydasını qorumaq üçün 'bu transaction-u təzədən hesabla' deməsidir.

Situasiya	Serializable uyğunluğu	Alternativ
Müəyyən saat üçün yalnız bir xüsusi kampaniya limiti	Uyğun ola bilər, xüsusilə bir çox sətir birlikdə qərar verirsə.	Atomic counter və ya constraint.
Balansdan məbləğ çıxmaq	Çox vaxt uyğun ola bilər, amma ledger dizaynı daha vacibdir.	Optimistic concurrency / ledger.
Eyni masa iki dəfə rezerv olunmasın	İşləyər, amma yeganə cavab deyil.	Hədəfli lock + availability check.
Instruction history yaratmaq	Lazımsızdır; bir neçə yazma icazəlidir.	ReadCommitted.

Serializable istifadə edildirsə, retry strategiyası mütləq olmalıdır. PostgreSQL serialization failure üçün çox vaxt SQLSTATE 40001 qaytarır.

Retry prinsipi

```
for (var attempt = 1; attempt <= 3; attempt++)
{
    try
    {
        return await RunBusinessTransactionAsync();
    }
    catch (PostgresException ex) when (ex.SqlState == "40001" && attempt < 3)
    {
        await Task.Delay(ShortBackoff(attempt));
    }
}

throw new BusinessException("Əməliyyat təkrar yoxlanmalıdır.");
```

7. Isolation, lock, constraint və outbox bir-birindən fərqlidir

Alət	Nəyi həll edir?	ECafe nümunəsi
Transaction	Bir neçə yazmanın hamısı və ya heç biri.	Instruction + notification + audit.
Isolation level	Paralel transaction-ların görünüş qaydası.	Payment instruction üçün ReadCommitted.
Lock	Konkret resursa paralel girişi növbəlayir.	Eyni masa üçün reservation lock.
Unique constraint	Database səviyyəsində təkrarı fiziki olaraq qadağan edir.	Məsələn, eyni unique biznes açarı.
Outbox	Yan effektin etibarlı, sonradan emalını təmin edir.	Audit event worker tərəfindən log-a çevrilir.
Idempotency key	Eyni HTTP sorğusunun təkrar emalını önlayir.	Payment gateway callback və ya create reservation request.

Outbox haqqında diqqət: ECafe audit log üçün outbox istifadə edir. Payment instruction endpoint-ində audit event transaction daxilində outbox-a düşür; sonra worker onu audit log-a çevirir. Bu yanaşma worker müvəqqəti dayansa belə audit məlumatının itməməsini hədəfləyir.

Həssas məlumat

Kart nömrəsi və ya bank hesabı audit metadata-da yazılmamalıdır. Auditdə instruction ID, rezervasiya ID, göndərən istifadəçi və məbləğ kifayətdir. Müştəriyə göstərilən mətn isə ayrıca payment instruction cədvəlində qalır.

8. ECafe üçün seçim bələdçisi

Flow	Təvsiyə	Səbəb
Rezervasiya yaratmaq	ReadCommitted + masa lock + availability yoxlaması	Eyni masa üçün paralel sorğular ardıcıl işləyir.
Payment instruction göndərmək	ReadCommitted + transaction + audit outbox + notification	Bir neçə instruction tarixçəsi normaldır; atomiklik lazımdır.
Payment proof yükləmək	ReadCommitted + ownership check + fayl access policy	Müştəri yalnız öz rezervasiyasına sübut yükləyə bilər.
Ödənişi təsdiqləmək	Transaction + status yoxlaması + idempotent transition	Eyni proof iki dəfə təsdiqlənməməlidir.
Yüksək konfliktli sayğac/limit	Serializable və ya atomic update	Çox sətirli qərarlar race condition yarada bilər.

Kod yazarkən yoxlama siyahısı

- Bu əməliyyatda qorunmalı biznes qaydası (invariant) nədir?
- Eyni request iki dəfə gələrsə nəticə nə olmalıdır? Idempotency lazımdır?
- Paralel sorğular eyni resursu dəyişirsə hədəfli lock və ya database constraint varmı?
- Transaction daxilində yalnız database yazmaları varmı? Xarici HTTP çağırışlarını commit-dən sonra outbox/worker ilə et.
- Serializable seçmisənsə, 40001 retry davranışı və test ssenarisi varmı?
- Audit metadata həssas məlumat daşıyırmı?

Bir cümləlik cavab

ECafe payment instruction endpoint-ində ReadCommitted doğrudur, çünki instruction history paralel yaradıla bilər; qorunan invariant atomiklikdir və transaction bunu təmin edir. Eyni masanın iki dəfə rezerv olunması isə ayrıca hədəfli masa lock-u ilə qorunur.

Sənəd sonu